

Data binding

Main content

- ❑ Basic Data binding
- ❑ Converter
- ❑ INotifyPropertyChanged
- ❑ List data binding: ComboBox, ListView

Basic Data binding

Use DataContext to create connection with the UI

```
private void Window_Loaded(object sender, RoutedEventArgs e)
{
    var sv = new Student {
        ID = "001",
        Fullname = "Nguyen Minh Tam"
    };
    this.DataContext = sv;
}
```

```
class Student
{
    0 references
    public string ID { get; set; }
    0 references
    public string Fullname { get; set; }
}
```

```
<Label Content="{Binding ID}" Width=100/>
<Label Content="{Binding Fullname}" Width=100/>
```

 Basic UI Demo

001

Nguyen Minh Tam

Minor adjustment

Let assume that, to graduate, a student must aggregate 360 credits

Let's add the current credits information

```
class Student
{
    1 reference
    public string ID { get; set; }
    1 reference
    public string Fullname { get; set; }
    0 references
    public int Credits { get; set; }
}
```

Editing the binding for the added property

```
private void Window_Loaded(object sender, RoutedEventArgs e)
{
    var sv = new Student {
        ID = "001",
        Fullname = "Nguyen Minh Tam",
        Credits = 80
    };
    this.DataContext = sv;
}
```

```
<Label Content="{Binding ID}" Width=
<Label Content="{Binding Fullname}"
<Label Content="{Binding Credits}" Width=
```

 Basic UI Demo

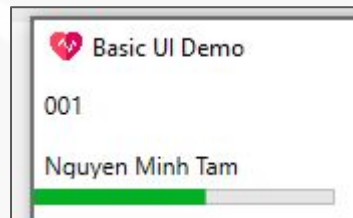
001

Nguyen Minh Tam

80

Changing way of displaying - ProgressBar

```
<Label Content="{Binding ID}" Width="150" Height="20" />
<Label Content="{Binding Fullname}" Width="150" Height="20" />
<ProgressBar Value="{Binding Credits}"
              Minimum="0" Maximum="140"
              Width="150" Height="8"
              Canvas.Top="55"/>
```



Better? Percentage? Circle?

<https://stackoverflow.com/questions/4871263/how-to-create-a-circular-style-progressbar>

Converter – Step 1

```
class CreditsToPercentageConverter : IValueConverter
{
    0 references
    public object Convert(object value, Type targetType)
    {
        int credits = (int) value ;
        double percentage = credits * 100 / 140;
        int number = (int)percentage;
        return $"{number} %";
    }
}
```

{number: 0.00}: allows two

Step 2: Declaration as a resource

```
<Window.Resources>  
    <local:CreditsToPercentageConverter x:Key="percentConverter" />  
</Window.Resources>
```

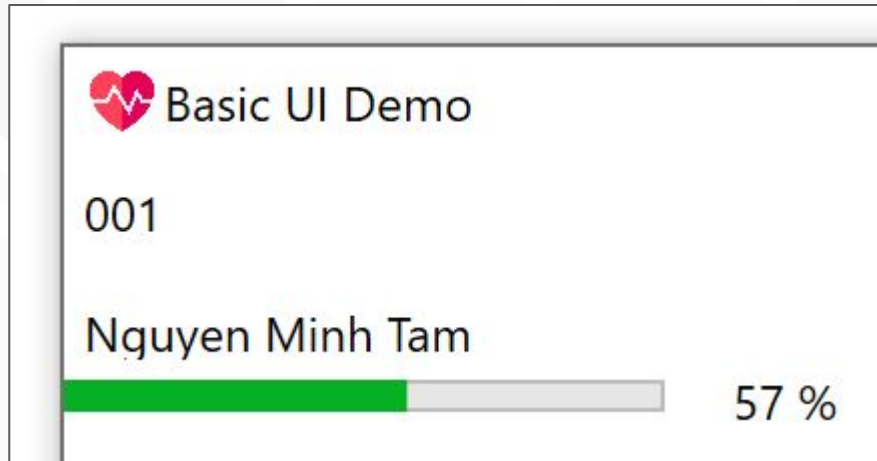
```
<Page.Resources>  
    <local:NameToNameWithLenConverter x:Key="converter"/>  
</Page.Resources>
```

The above is equivalent to this:

```
var percentConverter = new CreditsToPercentageConverter();
```

Step 3: Use the Converter

```
<TextBlock Text="{Binding Credits,  
Converter={StaticResource percentConverter}}"/>
```



Turn max into parameter

0 references

```
class CreditsToPercentageConverter : IValueConverter
{
    1 reference
    public int MaxCredits { get; set; }

    0 references
    public object Convert(object value, Type targetType,
    {
        int credits = (int) value ;
        double percentage = credits * 100 / MaxCredits;
        int number = (int)percentage;
        return $"{number} %";
    }
}
```

```
<Window.Resources>
    <local:CreditsToPercentageConverter
        x:Key="percentConverter" MaxCredits="140" />
</Window.Resources>
```

Multi value converter

C#

```
public class MyMultiValueConverter : IMultiValueConverter {  
    public object Convert(object[] values, Type targetType, object parameter, System.Globalization.CultureInfo culture) {  
        int firstValue = (int)values[0];  
        double secondValue = (double)values[1];  
        string thirdValue = (string)values[2];  
  
        return "You said " + thirdValue + ", but it's rather " + firstValue * secondValue;  
    }  
  
    public object[] ConvertBack(object value, Type[] targetTypes, object parameter, System.Globalization.CultureInfo culture) {  
        throw new NotImplementedException("Going back to what you had isn't supported.");  
    }  
}
```

XAML

```
<TextBlock.Text>  
    <MultiBinding Converter="{StaticResource myNs:MyMultiValueConverter}">  
        <Binding Path="FirstValue" />  
        <Binding Path="SecondValue" />  
        <Binding Path="ThirdValue" />  
    </MultiBinding>  
</TextBlock.Text>
```

In-class exercise – Project ConfigLoader

Create a class `Config` that stores:

- ❑ Last server
- ❑ Last username

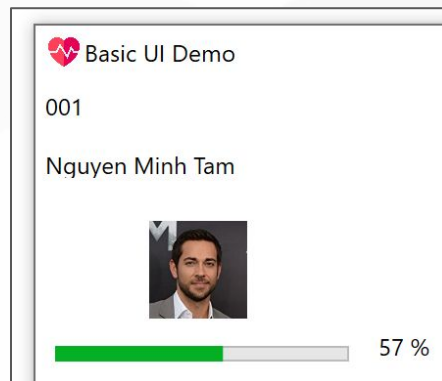
Create a screen that display that information using data binding

Displaying image using data binding

Using embeded image inside the executable file

```
class Student
{
    1 reference
    public string ID { get; set; }
    1 reference
    public string Fullname { get; set; }
    1 reference
    public int Credits { get; set; }
    0 references
    public string AvatarPath { get; set; }
}
```

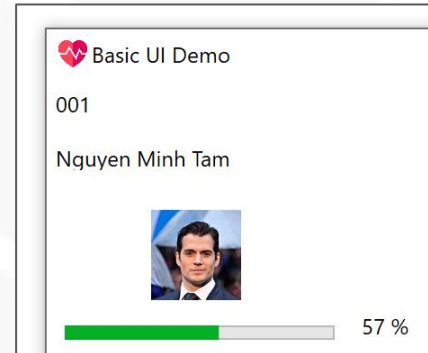
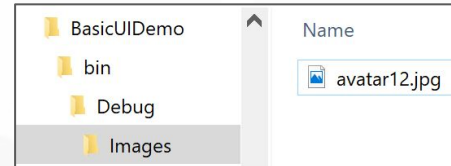
```
<Image Source="{Binding AvatarPath}"
        Width="50" Height="50"
        Canvas.Left="58" Canvas.Top="72"/>
```



Using absolute path

```
class RelativeToAbsolutePathConverter : IValueConverter
{
    1 reference
    public object Convert(object value, Type targetType, object parameter,
    {
        var imageFile = (string)value;
        var currentFolder = AppDomain.CurrentDomain.BaseDirectory;
        return currentFolder + imageFile;
    }
}
```

```
<local:RelativeToAbsolutePathConverter x:Key="absoluteConverter"/>
<Image Source="{Binding AvatarPath,
    Converter={StaticResource absoluteConverter}}"/>
```



Add rounded corner

```
<Border Width="50" Height="50" Canvas.Left="58" Canvas.Top="72"  
        CornerRadius="25">  
    <Border.Background>  
        <ImageBrush ImageSource="{Binding AvatarPath,  
            Converter={StaticResource absoluteConverter}}" />  
    </Border.Background>  
    <Border.Effect>  
        <DropShadowEffect />  
    </Border.Effect>  
</Border>
```



UI Synchronization

Core idea

Underlying *data* change => **UI** automatically change

Adding button to update

```
<Button Content="Finish" Name="finishButton"  
        Width="50" Height="25"  
        Click="finishButton_Click"  
        Canvas.Left="58" Canvas.Top="160"/>
```

001

Nguyen Minh Tam



Finish

Student _sv;

1 reference

```
private void Window_Loaded(object sender, RoutedEventArgs e)
{
    _sv = new Student {
        ID = "001",
        Fullname = "Nguyen Minh Tam",
        Credits = 80,
        AvatarPath = "Images/avatar12.jpg"
    };
    this.DataContext = _sv;
}
```

Update the credits when clicked

1 reference

```
private void finishButton_Click(object sender, RoutedEventArgs e)
{
    _sv.Credits = 140;
}
```

We see nothing change in the UI! Why?

Need to implement **INotifyPropertyChanged**

Underlying code of getter and setter

```
class Student
{
    1 reference
    public string ID { get; set; }
    1 reference
    public string Fullname { get; set; }

    private int _credits; // Backup field
    2 references
    public int Credits {
        get { return _credits; }
        set
        {
            _credits = value;
        }
    }
    1 reference
    public string AvatarPath { get; set; }
}
```

Implementing INotifyPropertyChanged

```
class Student: INotifyPropertyChanged
{
    1 reference
    public string ID { get; set; }
    1 reference
    public string Fullname { get; set; }

    public event PropertyChangedEventHandler PropertyChanged;

    private int _credits; // Backup field
    2 references
    public int Credits {
        get { return _credits; }
        set
        {
            _credits = value;
            PropertyChanged?.Invoke(this,
                new PropertyChangedEventArgs("Credits"));
        }
    }
}
```

Tools to use rather than manually writing

PropertyChanged.Fody (Need at least VS 2019)

Weaving: injecting functionality into an existing program

- ❑ Source code: inject source code lines before the code is compiled
- ❑ IL weaving (for .NET) adds the code as IL instructions in the assembly
- ❑ ByteCode weaving (for Java) works on the class file

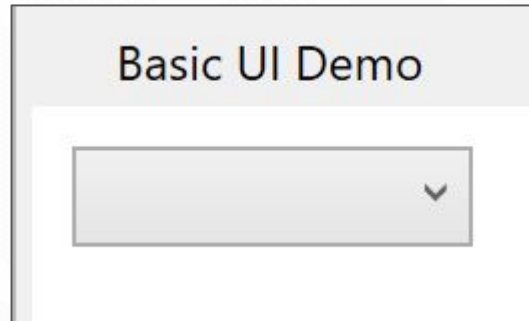
Homework

1. Do data binding for a **book**: Book's name, Cover's image, Author, Published Year, ISBN
2. Do data binding for a **mobile phone**: Phone's name, Image, Manufacturer, Price
3. Do data binding for an **employee**: Fullname, Email, Address, Telephone number, Avatar's image

Binding a list

Prepare UI using ComboBox

```
<Canvas>  
  <ComboBox Width="100" Height="25" Name="studentsComboBox"  
    Canvas.Left="10" Canvas.Top="10" >  
  </ComboBox>  
</Canvas>
```



Sample data

```
List<Student> _list;
```

1 reference

```
private void Window_Loaded(object sender, RoutedEventArgs e)
```

```
{
```

```
    _list = new List<Student>()
```

```
    {
```

```
        new Student() {ID = "001", Fullname="Nguyen Thanh Minh", Credits=32, AvatarPath="avatar11.jpg"},
```

```
        new Student() {ID = "001", Fullname="Tran Duc Long", Credits=12, AvatarPath="avatar12.jpg"},
```

```
        new Student() {ID = "001", Fullname="Do Huu Le", Credits=77, AvatarPath="avatar13.jpg"},
```

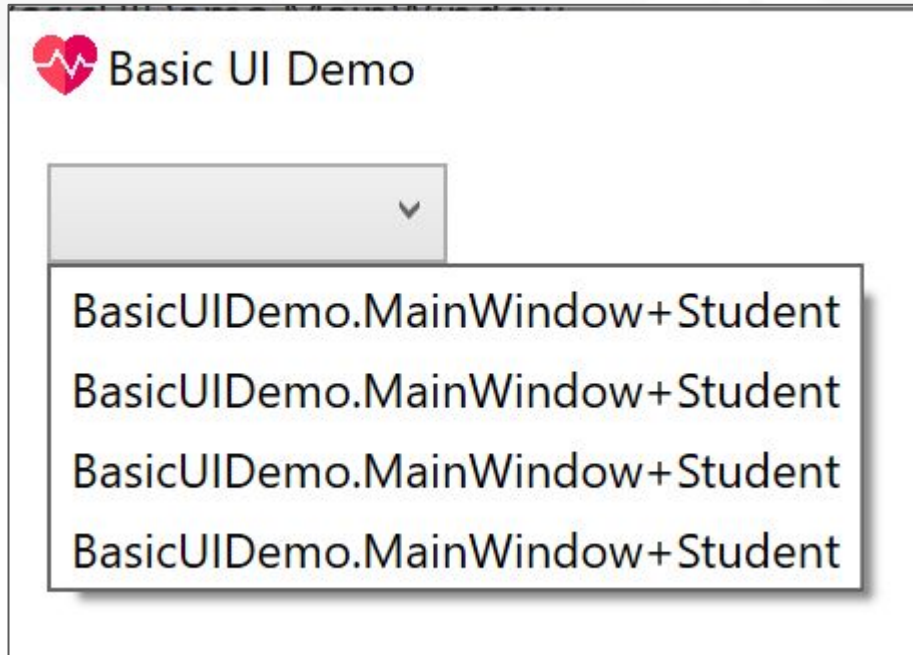
```
        new Student() {ID = "001", Fullname="Cao Thai Tuan", Credits=16, AvatarPath="avatar14.jpg"}
```

```
    };
```

```
    studentsComboBox.ItemsSource = _list;
```

```
}
```

The result



Simple way to modify

```
class Student: INotifyPropertyChanged
{
    0 references
    public override string ToString()
    {
        return $"{ID} - {Fullname}";
    }
}
```



But it should not be this way, why?

Because this class's job is not displaying data

Using ItemTemplate

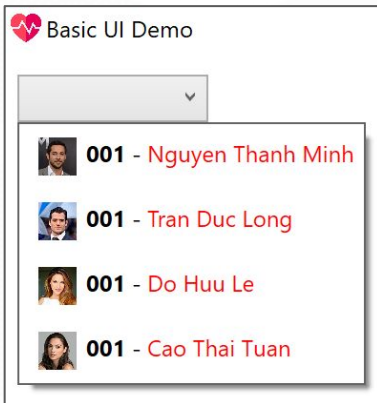
```
<ComboBox Width="100" Height="25" Name="studentsComboBox"
    Canvas.Left="10" Canvas.Top="10" >
    <ComboBox.ItemTemplate>
        <DataTemplate>
            <StackPanel Orientation="Horizontal">
                <TextBlock Text="{Binding ID}" FontWeight="Bold"/>
                <TextBlock Text=" - "/>
                <TextBlock Text="{Binding Fullname}" Foreground="Red"/>
            </StackPanel>
        </DataTemplate>
    </ComboBox.ItemTemplate>
</ComboBox>
```



001 - Nguyen Thanh Minh
001 - Tran Duc Long
001 - Do Huu Le
001 - Cao Thai Tuan

Adding image

```
<DataTemplate>
  <StackPanel Orientation="Horizontal" VerticalAlignment="Center">
    <Image Source="{Binding AvatarPath}" Width="20" Height="20" Margin="5"/>
    <TextBlock Text="{Binding ID}" FontWeight="Bold" Height="20"/>
    <TextBlock Text=" - " Height="20"/>
    <TextBlock Text="{Binding Fullname}" Foreground="Red" Height="20"/>
  </StackPanel>
</DataTemplate>
```

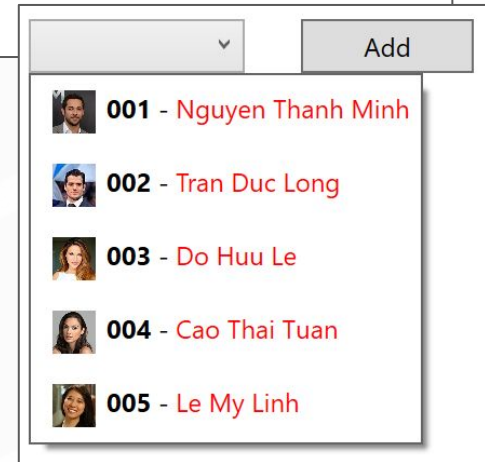


Editing the list

Adding a new item

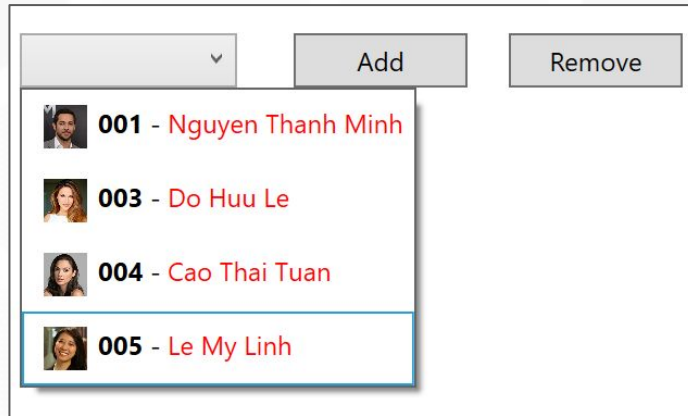
```
private void addStudentButton_Click(object sender, RoutedEventArgs e)
{
    _list.Add(new Student()
    {
        ID = "005",
        Fullname = "Le My Linh", Credits=44, AvatarPath="Images/avatar15.jpg"
    });
}
```

Convert the list to binding list



Delete an item

```
private void removeStudentButton_Click(object sender, RoutedEventArgs e)
{
    _list.RemoveAt(1);
}
```



Deleting an item - The other way

```
var student = (Student) studentsComboBox.SelectedItem;  
_students.Remove(student);
```

Update an item

```
private void updateStudentButton_Click(object sender, RoutedEventArgs e)
{
    _list[1].ID = "007";
    _list[1].Fullname = "James Bond";
}
```

Using ListView

Convert to using ListView

```
<ListView Width="200" Height="300" Name="studentsComboBox"
    Canvas.Left="10" Canvas.Top="10" >
    <ListView.ItemTemplate>
        <DataTemplate>
            <StackPanel Orientation="Horizontal" VerticalAlignment="Center">
                <Image Source="{Binding AvatarPath}" Width="20" Height="20" Margin="5"/>
                <TextBlock Text="{Binding ID}" FontWeight="Bold" Height="20"/>
                <TextBlock Text=" - " Height="20"/>
                <TextBlock Text="{Binding Fullname}" Foreground="Red" Height="20"/>
            </StackPanel>
        </DataTemplate>
    </ListView.ItemTemplate>
</ListView>
```

 Basic UI Demo



001 - Nguyen Thanh Minh

Add

Remove



002 - Tran Duc Long

Update



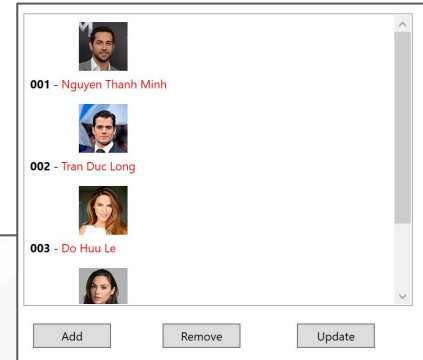
003 - Do Huu Le



004 - Cao Thai Tuan

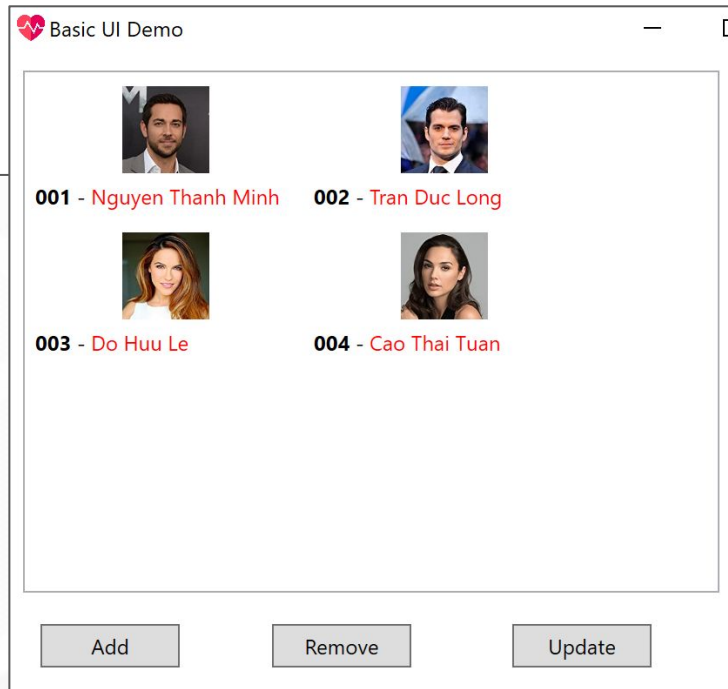
Minor change in displaying

```
<ListView Width="400" Height="300" Name="studentsComboBox"
    Canvas.Left="10" Canvas.Top="10" >
    <ListView.ItemTemplate>
        <DataTemplate>
            <StackPanel VerticalAlignment="Center" Width="150" Height="80">
                <Image Source="{Binding AvatarPath}" Width="50" Height="50" Margin="5"/>
                <StackPanel Orientation="Horizontal" >
                    <TextBlock Text="{Binding ID}" FontWeight="Bold" Height="20"/>
                    <TextBlock Text=" - " Height="20"/>
                    <TextBlock Text="{Binding Fullname}" Foreground="Red" Height="20"/>
                </StackPanel>
            </StackPanel>
        </DataTemplate>
    </ListView.ItemTemplate>
</ListView>
```



Using WrapPanel

```
<ListView Width="400" Height="300" Name="studentsComboBox"
    Canvas.Left="10" Canvas.Top="10" ScrollViewer.HorizontalScrollBarVisibility="Disabled">
    <ListView.ItemsPanel>
        <ItemsPanelTemplate>
            <WrapPanel />
        </ItemsPanelTemplate>
    </ListView.ItemsPanel>
```



Homework

1. Do data binding for a list of **books**: Book's name, Cover's image, Author, Published Year
2. Do data binding for a list of **mobile phones**: Phone's name, Image, Manufacturer, Price
3. Do data binding for a list of **employees**: Fullname, Email, Address, Telephone number, Avatar's image

Prepare at least 10 items

Add, Delete, Update (hard code data)

Saving template for reuse - Step 01

- ❑ Extract data template to a **Dictionary.xaml** file

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winxaml/2006"
                    xmlns:x="http://schemas.microsoft.com/winxaml/2006">
    <DataTemplate x:Key="StudentThumbnailView">
        <StackPanel Width="150" Height="120">
            <Image Width="70" Height="70" Source="{Binding Image}" />
            <TextBlock>
                <Run Text="{Binding ID}" Foreground="Red" />
                <Run Text="{Binding Name}" FontWeight="Bold" />
            </TextBlock>
            <StackPanel Orientation="Horizontal">
                <TextBlock Text="{Binding Credits}" />
                <ProgressBar Minimum="0" Maximum="100" Value="{Binding Credits}" />
            </StackPanel>
        </StackPanel>
    </DataTemplate>
</ResourceDictionary>
```

Extracting template - Step 02

Merge resource into our Window

```
<Window.Resources>  
  <ResourceDictionary Source="StudentDictionary.xaml"/>  
</Window.Resources>
```

Extracting template - Step 03

Specify the template we want to use

```
<ListView Name="studentsComboBox"  
    ItemTemplate="{StaticResource StudentThumbnailView}"
```

Handling context menu

```
<ListView.Resources>
    <ContextMenu x:Key="TenCuaContextMenu">
        <MenuItem Header="Delete" Click="MenuItem_Click"/>
        <MenuItem Header="Edit"/>
    </ContextMenu>
</ListView.Resources>
<ListView.ItemContainerStyle>
    <Style TargetType="{x:Type ListViewItem}" >
        <Setter Property="ContextMenu" Value="{StaticResource
TenCuaContextMenu}" />
    </Style>
</ListView.ItemContainerStyle>
```

Handling double click

Does not work when clicking on empty space, must set event handler using Style

```
<ListView.ItemContainerStyle>
  <Style TargetType="{x:Type ListViewItem}">
    <Setter Property="ContextMenu"
      Value="{StaticResource listViewContextMenu}"/>
    <EventSetter Event="MouseDoubleClick"
      Handler="listViewItem_DoubleClick"/>
  </Style>
</ListView.ItemContainerStyle>
```


Master-Detail data binding

```
<ComboBox Name="studentsComboBox" Width="200" Height="35" Canvas.Left="38" Canvas.Top="89">
    <ComboBox.ItemTemplate...>
</ComboBox>
<StackPanel Canvas.Left="290" Canvas.Top="89"
    DataContext="{Binding ElementName=studentsComboBox,Path=SelectedItem}">
    <TextBlock Text="{Binding ID}" d:Text="001"/>
    <TextBlock Text="{Binding Name}" d:Text="Tran Huy Hoang"/>
</StackPanel>
```

Tran Van An ▼

001
Tran Van An

Add